Engineer to Excel

NATURAL LANGUAGE PROCESSING LABORATORY

LAB RECORD

DSA0303 - NATURAL LANGUAGE PROCESSING

Dec 2026 (UG) • SLOT A

Implementation of Fundamental NLP Techniques using Python (Google Colab)

Name	: B Tharun
Register Number	: 1924260010
Department	: AIDS
Year	: 1st
Subject Code	: DSA0303
Faculty	: Akshaya K

INDEX

Ex. No.	Date	Name of the Experiment	Page No.	Signature
11		Simple Top-Down Parser For Context-Free Grammars	3	
12		Earley Parser For Context-Free Grammars	6	
13		Parse Tree Generation Using A Context-Free Grammar	9	
14		Agreement Checking Using A Context-Free Grammar	12	
15		Probabilistic Context-Free Grammar (Pcfg) Parsing	14	
16		Named Entity Recognition (Ner) Using Spacy	17	
17		Accessing Wordnet: Synsets And Word Meanings	19	
18		Simple Fopc (First-Order Predicate Calculus) Parser	21	
19		Word Sense Disambiguation Using The Lesk Algorithm	25	
20		Information Retrieval Using Tf-Idf	28	
21		Syntax-Driven Semantic Analysis	30	
22		Reference Resolution	32	
23		Coherence Evaluation Of A Text	34	
24		Dialog Act Recognition	36	
25		Text Generation Using Gpt-3	38	
26		Machine Translation Using Hugging Face Transformers	40	

EXPERIMENT NO: 11

SIMPLE TOP-DOWN PARSER FOR CONTEXT-FREE GRAMMARS

Date: _____

AIM:

To implement a simple top-down (recursive-descent with backtracking) parser that decides whether a sentence is generated by a context-free grammar (CFG) and builds its parse tree.

ALGORITHM:

Step 1: Start the program.

Step 2: Represent the CFG as a dictionary that maps every non-terminal to the list of its productions.

Step 3: Define parse(symbol, words, pos): if the symbol is a terminal, compare it with words[pos] and advance the position on a match.

Step 4: If the symbol is a non-terminal, try each of its productions in order and parse the right-hand-side symbols from left to right (parse_seq); backtrack to the next production whenever a symbol fails.

Step 5: Start the derivation with the start symbol S at position 0 and accept the sentence only if a derivation consumes every word.

Step 6: Print the expansion trace, and then the parse tree of an accepted sentence or a rejection message.

Step 7: Stop the program.

CODE:

```
# Simple Top-Down (Recursive Descent with Backtracking) Parser for a CFG
grammar = {
    "S":   [["NP", "VP"]],
    "NP":  [["Det", "N"], ["Det", "Adj", "N"], ["PN"]],
    "VP":  [["V", "NP"], ["V"]],
    "Det": [["the"], ["a"]],
    "Adj": [["big"], ["small"]],
    "N":   [["dog"], ["cat"], ["ball"]],
    "V":   [["chased"], ["saw"], ["sleeps"]],
    "PN":  [["John"], ["Mary"]],
}

def parse(symbol, words, pos, depth=0, trace=False):
    """Yield (tree, next_position) for every way `symbol` can derive words[pos:...]."""
    if symbol not in grammar:                       # terminal symbol
        if pos < len(words) and words[pos] == symbol:
            yield symbol, pos + 1
        return
    for production in grammar[symbol]:
        if trace:
            print("  " * depth + f"try {symbol} -> {' '.join(production)}")
        for children, new_pos in parse_seq(production, words, pos, depth + 1, trace):
            yield (symbol, children), new_pos

def parse_seq(symbols, words, pos, depth, trace):
    if not symbols:
        yield [], pos
        return
    for tree, p in parse(symbols[0], words, pos, depth, trace):
        for rest, p2 in parse_seq(symbols[1:], words, p, depth, trace):
            yield [tree] + rest, p2

def show(tree, indent=0):
    if isinstance(tree, str):
        print("  " * indent + tree); return
    print("  " * indent + tree[0])
    for child in tree[1]:
        show(child, indent + 1)

def top_down(sentence, trace=False):
    words = sentence.split()
    for tree, end in parse("S", words, 0, trace=trace):
        if end == len(words):                       # whole input consumed
            return tree
    return None

print("Trace of top-down expansion for: 'John saw a cat'")
top_down("John saw a cat", trace=True)

for s in ["the big dog chased a cat", "Mary sleeps", "dog the chased cat"]:
    print("\nSentence:", s)
    t = top_down(s)
    if t:
        print("ACCEPTED - parse tree:"); show(t)
    else:
        print("REJECTED - sentence is not generated by the grammar")
```

OUTPUT:

```
Trace of top-down expansion for: 'John saw a cat'
try S -> NP VP
  try NP -> Det N
    try Det -> the
    try Det -> a
  try NP -> Det Adj N
    try Det -> the
    try Det -> a
  try NP -> PN
    try PN -> John
  try VP -> V NP
    try V -> chased
    try V -> saw
    try NP -> Det N
      try Det -> the
      try Det -> a
      try N -> dog
      try N -> cat

Sentence: the big dog chased a cat
ACCEPTED - parse tree:
S
  NP
    Det
      the
    Adj
      big
    N
      dog
  VP
    V
      chased
    NP
      Det
        a
      N
        cat

Sentence: Mary sleeps
ACCEPTED - parse tree:
S
  NP
    PN
      Mary
  VP
    V
      sleeps

Sentence: dog the chased cat
REJECTED - sentence is not generated by the grammar
```

RESULT:

The top-down trace shows S being expanded to NP VP first and the parser backtracking over alternative productions (for example Det -> the, then Det -> a, and V -> chased, then V -> saw). The sentences "the big dog chased a cat" and "Mary sleeps" were accepted and their parse trees were printed, whereas "dog the chased cat" was rejected. Thus the top-down parser for the context-free grammar was implemented successfully.

EXPERIMENT NO: 12

EARLEY PARSER FOR CONTEXT-FREE GRAMMARS

Date: _____

AIM:

To implement an Earley chart parser that recognises sentences of a context-free grammar using the predict, scan and complete operations.

ALGORITHM:

Step 1: Start the program.

Step 2: Define the grammar as a dictionary of productions and create one chart column (state set) for every position of the sentence, 0 to n.

Step 3: Add the dummy start state GAMMA -> • S [0] to Chart[0]; each state is a tuple (lhs, rhs, dot, origin).

Step 4: For every state in Chart[i]: if the symbol after the dot is a non-terminal, apply PREDICT and add its productions with origin i.

Step 5: If the symbol after the dot is a terminal that equals the next word, apply SCAN and add the state with the dot advanced to Chart[i+1].

Step 6: If the dot is at the end of the rule, apply COMPLETE: advance the dot of every state in Chart[origin] that was waiting for this non-terminal.

Step 7: Print the completed states of every chart column.

Step 8: Accept the sentence if GAMMA -> S • [0] is present in the last chart column; stop the program.

CODE:

```
# Earley Parser for a Context-Free Grammar
grammar = {
    "S":   [["NP", "VP"]],
    "NP":  [["Det", "N"], ["NP", "PP"], ["John"]],
    "VP":  [["V", "NP"], ["VP", "PP"]],
    "PP":  [["P", "NP"]],
    "Det": [["the"], ["a"]],
    "N":   [["man"], ["dog"], ["telescope"]],
    "V":   [["saw"]],
    "P":   [["with"]],
}

def earley(words, start="S"):
    chart = [[] for _ in range(len(words) + 1)]
    def add(i, state):
        if state not in chart[i]:
            chart[i].append(state)
    add(0, ("GAMMA", (start,), 0, 0))              # state = (lhs, rhs, dot, origin)
    for i in range(len(words) + 1):
        j = 0
        while j < len(chart[i]):
            lhs, rhs, dot, origin = chart[i][j]; j += 1
            if dot < len(rhs):
                nxt = rhs[dot]
                if nxt in grammar:                                  # PREDICT
                    for prod in grammar[nxt]:
                        add(i, (nxt, tuple(prod), 0, i))
                elif i < len(words) and words[i] == nxt:            # SCAN
                    add(i + 1, (lhs, rhs, dot + 1, origin))
            else:                                                   # COMPLETE
                for l2, r2, d2, o2 in list(chart[origin]):
                    if d2 < len(r2) and r2[d2] == lhs:
                        add(i, (l2, r2, d2 + 1, o2))
    return chart

def fmt(state):
    lhs, rhs, dot, origin = state
    r = list(rhs); r.insert(dot, "•")
    return f"{lhs} -> {' '.join(r)}  [{origin}]"

sentence = "John saw the man with a telescope".split()
chart = earley(sentence)
for i, states in enumerate(chart):
    word = sentence[i - 1] if i else "(start)"
    print(f"\n=== Chart[{i}]  after '{word}'  ({len(states)} states) ===")
    for s in states:
        if s[2] == len(s[1]) or i == 0:        # show completed states (and all of chart 0)
            print("  ", fmt(s))

accepted = ("GAMMA", ("S",), 1, 0) in chart[-1]
print("\nSentence accepted by grammar:", accepted)
```

OUTPUT:

```
=== Chart[0]  after '(start)'  (7 states) ===
   GAMMA -> • S  [0]
   S -> • NP VP  [0]
   NP -> • Det N  [0]
   NP -> • NP PP  [0]
   NP -> • John  [0]
   Det -> • the  [0]
   Det -> • a  [0]

=== Chart[1]  after 'John'  (8 states) ===
   NP -> John •  [0]

=== Chart[2]  after 'saw'  (7 states) ===
   V -> saw •  [1]

=== Chart[3]  after 'the'  (5 states) ===
   Det -> the •  [2]

=== Chart[4]  after 'man'  (9 states) ===
   N -> man •  [3]
   NP -> Det N •  [2]
   VP -> V NP •  [1]
   S -> NP VP •  [0]
   GAMMA -> S •  [0]

=== Chart[5]  after 'with'  (7 states) ===
   P -> with •  [4]

=== Chart[6]  after 'a'  (5 states) ===
   Det -> a •  [5]

=== Chart[7]  after 'telescope'  (13 states) ===
   N -> telescope •  [6]
   NP -> Det N •  [5]
   PP -> P NP •  [4]
   NP -> NP PP •  [2]
   VP -> VP PP •  [1]
   VP -> V NP •  [1]
   S -> NP VP •  [0]
   GAMMA -> S •  [0]

Sentence accepted by grammar: True
```

RESULT:

The chart for "John saw the man with a telescope" was built for all 8 positions. The last column contains GAMMA -> S • [0], so the sentence was accepted. Both NP -> NP PP • [2] and VP -> VP PP • [1] appear as completed states in Chart[7], showing the two possible attachments of the phrase "with a telescope". Thus the Earley parser was implemented successfully.

EXPERIMENT NO: 13

PARSE TREE GENERATION USING A CONTEXT-FREE GRAMMAR

Date: _____

AIM:

To generate and display the parse tree(s) of a given sentence using a context-free grammar and the chart parser of the NLTK library.

ALGORITHM:

Step 1: Start the program.

Step 2: Import nltk and CFG from nltk.

Step 3: Define the grammar with CFG.fromstring(), giving the S, NP, VP, PP rules and the lexical rules for Det, N, V and P.

Step 4: Create a ChartParser object from the grammar.

Step 5: Split the input sentence into words and call parser.parse() to obtain all the possible trees.

Step 6: Print the number of trees and, for every tree, its bracketed form and its drawn form using pretty_print().

Step 7: Stop the program.

CODE:

```
# Generate parse trees for a sentence using a Context-Free Grammar (NLTK)
import nltk
from nltk import CFG

grammar = CFG.fromstring("""
S  -> NP VP
NP -> Det N | Det N PP | 'I'
VP -> V NP | VP PP
PP -> P NP
Det -> 'the' | 'a'
N  -> 'man' | 'telescope' | 'park'
V  -> 'saw'
P  -> 'with' | 'in'
""")

parser = nltk.ChartParser(grammar)
sentence = "I saw the man with a telescope".split()

trees = list(parser.parse(sentence))
print("Sentence :", " ".join(sentence))
print("Number of parse trees:", len(trees))
for i, tree in enumerate(trees, 1):
    print(f"\n--- Parse tree {i} ---")
    print(tree)                 # bracketed form
    tree.pretty_print()         # drawn tree
```

OUTPUT:

```
Sentence : I saw the man with a telescope
Number of parse trees: 2

--- Parse tree 1 ---
(S
  (NP I)
  (VP
    (VP (V saw) (NP (Det the) (N man)))
    (PP (P with) (NP (Det a) (N telescope)))))
     S
  ___|____________
 |               VP
 |        _______|________
 |       VP               PP
 |    ___|___          ____|___
 |   |       NP       |        NP
 |   |    ___|___     |     ___|______
 NP  V  Det      N    P   Det         N
 |   |   |       |    |    |          |
 I  saw the     man with  a      telescope


--- Parse tree 2 ---
(S
  (NP I)
  (VP
    (V saw)
    (NP (Det the) (N man) (PP (P with) (NP (Det a) (N telescope))))))
     S
  ___|_______
 |           VP
 |    _______|___
 |   |           NP
 |   |    _______|____
 |   |   |   |        PP
 |   |   |   |    ____|___
 |   |   |   |   |        NP
 |   |   |   |   |     ___|______
 NP  V  Det  N   P   Det         N
 |   |   |   |   |    |          |
 I  saw the man with  a      telescope
```

RESULT:

The sentence "I saw the man with a telescope" produced 2 parse trees. In the first tree the phrase "with a telescope" is attached to the verb phrase (the seeing was done with a telescope), and in the second it is attached to the noun phrase (the man has the telescope). Thus parse trees were generated successfully and the structural ambiguity of the sentence was demonstrated.

EXPERIMENT NO: 14

AGREEMENT CHECKING USING A CONTEXT-FREE GRAMMAR

Date: _____

AIM:

To check subject-verb and determiner-noun number agreement in sentences using a feature-based context-free grammar, in which the NUM feature must unify across the constituents.

ALGORITHM:

Step 1: Start the program.

Step 2: Import FeatureGrammar and FeatureChartParser from nltk.

Step 3: Write the grammar so that the rules share one variable ?n for the NUM feature, e.g. S -> NP[NUM=?n] VP[NUM=?n] and NP[NUM=?n] -> Det[NUM=?n] N[NUM=?n].

Step 4: Add lexical entries with NUM=sg or NUM=pl for the determiners, nouns and verbs.

Step 5: For each test sentence, convert it to lower case, split it into words and run the feature chart parser.

Step 6: If at least one parse exists, report AGREEMENT OK; if no parse exists, report AGREEMENT VIOLATION.

Step 7: Print the result for all the test sentences and stop the program.

CODE:

```
# Check subject-verb / determiner-noun agreement using a feature-based CFG
from nltk.grammar import FeatureGrammar
from nltk.parse import FeatureChartParser

grammar = FeatureGrammar.fromstring("""
% start S
S -> NP[NUM=?n] VP[NUM=?n]
NP[NUM=?n] -> Det[NUM=?n] N[NUM=?n]
NP[NUM=pl] -> N[NUM=pl]
VP[NUM=?n] -> V[NUM=?n] NP[NUM=?m]
Det[NUM=sg] -> 'this' | 'a'
Det[NUM=pl] -> 'these' | 'many'
Det[NUM=?n] -> 'the'
N[NUM=sg] -> 'dog' | 'cat'
N[NUM=pl] -> 'dogs' | 'cats'
V[NUM=sg] -> 'chases' | 'sees'
V[NUM=pl] -> 'chase' | 'see'
""")
parser = FeatureChartParser(grammar)

def check(sentence):
    words = sentence.lower().split()
    try:
        trees = list(parser.parse(words))
    except ValueError as e:                    # word not in grammar
        return "UNKNOWN WORD"
    return "AGREEMENT OK" if trees else "AGREEMENT VIOLATION"

tests = ["the dog chases a cat", "these dogs chase the cat", "the dogs chases a cat",
         "this dogs see a cat", "a cat sees these dogs", "these cat chase a dog",
         "many dogs see the cats"]
print(f"{'Sentence':<30}| Result")
print("-" * 55)
for s in tests:
    print(f"{s:<30}| {check(s)}")
```

OUTPUT:

```
Sentence                      | Result
-------------------------------------------------------
the dog chases a cat          | AGREEMENT OK
these dogs chase the cat      | AGREEMENT OK
the dogs chases a cat         | AGREEMENT VIOLATION
this dogs see a cat           | AGREEMENT VIOLATION
a cat sees these dogs         | AGREEMENT OK
these cat chase a dog         | AGREEMENT VIOLATION
many dogs see the cats        | AGREEMENT OK
```

RESULT:

Out of 7 test sentences, 4 satisfied agreement ("the dog chases a cat", "these dogs chase the cat", "a cat sees these dogs", "many dogs see the cats") and 3 were flagged as violations ("the dogs chases a cat", "this dogs see a cat", "these cat chase a dog"). Thus agreement checking was implemented successfully using feature unification in a CFG.

EXPERIMENT NO: 15

PROBABILISTIC CONTEXT-FREE GRAMMAR (PCFG) PARSING

Date: _____

AIM:

To parse a sentence with a probabilistic context-free grammar and to select the most probable parse tree using the inside chart parser and the Viterbi parser of NLTK.

ALGORITHM:

Step 1: Start the program.

Step 2: Import PCFG, ViterbiParser and InsideChartParser from nltk.

Step 3: Define the PCFG with PCFG.fromstring(); the probabilities of the productions of every non-terminal must add up to 1.

Step 4: Parse the sentence with InsideChartParser to obtain all the trees, and sort them in decreasing order of probability (tree.prob()).

Step 5: Print every tree with its probability, i.e. the product of the probabilities of the rules used in it.

Step 6: Run the ViterbiParser to obtain the single most probable parse.

Step 7: Print the best tree and its probability, and stop the program.

CODE:

```
# Probabilistic Context-Free Grammar (PCFG) parsing
import nltk
from nltk import PCFG
from nltk.parse import ViterbiParser, InsideChartParser

grammar = PCFG.fromstring("""
S   -> NP VP        [1.0]
PP  -> P NP         [1.0]
NP  -> Det N        [0.5] | NP PP [0.2] | 'John' [0.3]
VP  -> V NP         [0.6] | VP PP [0.4]
Det -> 'the'        [0.6] | 'a'   [0.4]
N   -> 'man'        [0.4] | 'telescope' [0.3] | 'dog' [0.3]
V   -> 'saw'        [1.0]
P   -> 'with'       [1.0]
""")

sentence = "John saw the man with a telescope".split()
print("Sentence:", " ".join(sentence))

print("\nAll parses ranked by probability (inside chart parser):")
trees = sorted(InsideChartParser(grammar).parse(sentence), key=lambda t: -t.prob())
for i, t in enumerate(trees, 1):
    print(f"\nParse {i}   P = {t.prob():.6f}")
    t.pretty_print()

best = next(ViterbiParser(grammar).parse(sentence))
print("Most probable parse (Viterbi):")
print(best)
print("Probability =", round(best.prob(), 6))
```

OUTPUT:

```
Sentence: John saw the man with a telescope

All parses ranked by probability (inside chart parser):

Parse 1   P = 0.000518
      S
  ____|____________
 |                 VP
 |         ________|________
 |        VP                PP
 |     ___|___          ____|___
 |    |       NP      |        NP
 |    |    ___|___    |     ___|______
 NP   V  Det      N   P   Det         N
 |    |   |       |   |    |          |
John saw the     man with  a      telescope


Parse 2   P = 0.000259
      S
  ____|____________
 |                 VP
 |     ____________|___
 |    |                NP
 |    |        ________|____
 |    |       |             PP
 |    |       |         ____|___
 |    |       NP      |        NP
 |    |    ___|___    |     ___|______
 NP   V  Det      N   P   Det         N
 |    |   |       |   |    |          |
John saw the     man with  a      telescope

Most probable parse (Viterbi):
(S
  (NP John)
  (VP
    (VP (V saw) (NP (Det the) (N man)))
    (PP (P with) (NP (Det a) (N telescope))))) (p=0.0005184)
Probability = 0.000518
```

RESULT:

The sentence "John saw the man with a telescope" has two parses. The tree in which the prepositional phrase is attached to the verb phrase has probability 0.000518 and the one in which it is attached to the noun phrase has probability 0.000259, so the Viterbi parser returned the verb-phrase attachment as the most probable parse. Thus PCFG parsing was implemented successfully.

EXPERIMENT NO: 16

NAMED ENTITY RECOGNITION (NER) USING SPACY

Date: _____

AIM:

To identify and classify named entities such as persons, organisations, places, dates and money in a text using the pre-trained statistical model of the spaCy library.

ALGORITHM:

Step 1: Start the program.

Step 2: Import spacy and load the English model with spacy.load("en_core_web_sm").

Step 3: Store the input text in the variable text and process it with nlp(text) to create a Doc object.

Step 4: Iterate over doc.ents and print every entity with its label and the description obtained from spacy.explain().

Step 5: Count the entities of every label using collections.Counter and print the counts.

Step 6: Stop the program.

CODE:

```
# Named Entity Recognition (NER) using spaCy
import spacy
nlp = spacy.load("en_core_web_sm")

text = ("Apple Inc. was founded by Steve Jobs in California in 1976. "
        "Sundar Pichai, the CEO of Google, visited Mumbai on 5 March 2024 "
        "and met officials of the Government of India. Tesla plans to invest $2 billion in Texas.")

doc = nlp(text)
print(f"{'Entity':<28}{'Label':<10}Description")
print("-" * 70)
for ent in doc.ents:
    print(f"{ent.text:<28}{ent.label_:<10}{spacy.explain(ent.label_)}")

from collections import Counter
print("\nEntity count by label:", dict(Counter(e.label_ for e in doc.ents)))
```

OUTPUT:

```
Entity                      Label     Description
----------------------------------------------------------------------
Apple Inc.                  ORG       Companies, agencies, institutions, etc.
Steve Jobs                  PERSON    People, including fictional
California                  GPE       Countries, cities, states
1976                        DATE      Absolute or relative dates or periods
Sundar Pichai               PERSON    People, including fictional
Google                      ORG       Companies, agencies, institutions, etc.
Mumbai                      GPE       Countries, cities, states
5 March 2024                DATE      Absolute or relative dates or periods
the Government of India     ORG       Companies, agencies, institutions, etc.
$2 billion                  MONEY     Monetary values, including unit
Texas                       GPE       Countries, cities, states

Entity count by label: {'ORG': 3, 'PERSON': 2, 'GPE': 3, 'DATE': 2, 'MONEY': 1}
```

RESULT:

The spaCy model recognised 11 named entities in the text: 3 organisations (Apple Inc., Google, the Government of India), 2 persons (Steve Jobs, Sundar Pichai), 3 geopolitical entities (California, Mumbai, Texas), 2 dates (1976, 5 March 2024) and 1 monetary value ($2 billion). Thus named entity recognition was implemented successfully using spaCy.

EXPERIMENT NO: 17

ACCESSING WORDNET: SYNSETS AND WORD MEANINGS

Date: _____

AIM:

To access WordNet, a lexical database of English, and to retrieve synsets, definitions, examples, synonyms, antonyms, hypernyms and the similarity of words using the NLTK library.

ALGORITHM:

Step 1: Start the program.

Step 2: Import nltk and wordnet from nltk.corpus and download the wordnet data.

Step 3: Call wn.synsets("bank") to list all senses of the word, and print the name, part of speech, definition and an example of the first five.

Step 4: Take the synset dog.n.01 and print its definition, lemma names, hypernyms, hyponyms and its hypernym path from the root.

Step 5: Collect the synonyms and the antonyms of the adjective "good" from the lemmas of its synsets.

Step 6: Compute the path similarity between dog and cat and between dog and car.

Step 7: Stop the program.

CODE:

```
# Accessing WordNet: synsets, definitions, relations
import nltk
from nltk.corpus import wordnet as wn
nltk.download("wordnet", quiet=True)

word = "bank"
syns = wn.synsets(word)
print(f"'{word}' has {len(syns)} synsets. First five:")
for s in syns[:5]:
    print(f"  {s.name():<28} [{s.pos()}] {s.definition()}")
    if s.examples():
        print(f"{'':<31}e.g. \"{s.examples()[0]}\"")

dog = wn.synset("dog.n.01")
print("\nSynset        :", dog.name())
print("Definition    :", dog.definition())
print("Lemmas        :", dog.lemma_names())
print("Hypernyms     :", sorted(h.name() for h in dog.hypernyms()))
print("Hyponyms (3)  :", sorted(h.name() for h in dog.hyponyms())[:3])
print("Hypernym path :", " > ".join(s.name().split('.')[0] for s in min(dog.hypernym_paths(), key=len)))

print("\nSynonyms of 'good' (adj):", sorted({l.name() for s in wn.synsets("good", "a") for l in s.lemmas()})
[:8])
ant = {a.name() for s in wn.synsets("good", "a") for l in s.lemmas() for a in l.antonyms()}
print("Antonyms of 'good'      :", sorted(ant))

cat = wn.synset("cat.n.01")
print("\nSimilarity (path) dog-cat :", round(dog.path_similarity(cat), 3))
print("Similarity (path) dog-car :", round(dog.path_similarity(wn.synset("car.n.01")), 3))
```

OUTPUT:

```
'bank' has 18 synsets. First five:
  bank.n.01                    [n] sloping land (especially the slope beside a body of water)
                               e.g. "they pulled the canoe up on the bank"
  depository_financial_institution.n.01 [n] a financial institution that accepts deposits and channels the
money into lending activities
                               e.g. "he cashed a check at the bank"
  bank.n.03                    [n] a long ridge or pile
                               e.g. "a huge bank of earth"
  bank.n.04                    [n] an arrangement of similar objects in a row or in tiers
                               e.g. "he operated a bank of switches"
  bank.n.05                    [n] a supply or stock held in reserve for future use (especially in emergencies)

Synset        : dog.n.01
Definition    : a member of the genus Canis (probably descended from the common wolf) that has been
domesticated by man since prehistoric times; occurs in many breeds
Lemmas        : ['dog', 'domestic_dog', 'Canis_familiaris']
Hypernyms     : ['canine.n.02', 'domestic_animal.n.01']
Hyponyms (3)  : ['basenji.n.01', 'corgi.n.01', 'cur.n.01']
Hypernym path : entity > physical_entity > object > whole > living_thing > organism > animal > domestic_animal
> dog

Synonyms of 'good' (adj): ['adept', 'beneficial', 'dear', 'dependable', 'effective', 'estimable', 'expert',
'full']
Antonyms of 'good'      : ['bad', 'evil']

Similarity (path) dog-cat : 0.2
Similarity (path) dog-car : 0.077
```

RESULT:

The word "bank" has 18 synsets (for example the slope beside a river and a financial institution). The synset dog.n.01 has the hypernyms domestic_animal and canine, and its hypernym path runs from entity down to dog. The antonyms of "good" were found to be "bad" and "evil", and the path similarity of dog and cat (0.2) was higher than that of dog and car (0.077). Thus WordNet was explored successfully.

EXPERIMENT NO: 18

SIMPLE FOPC (FIRST-ORDER PREDICATE CALCULUS) PARSER

Date: _____

AIM:

To implement a simple recursive-descent parser for basic first-order logic expressions that builds an abstract syntax tree (AST) and finds the free variables of the formula.

ALGORITHM:

Step 1: Start the program.

Step 2: Define a regular expression and a tokenize() function to split the input into the tokens ->, &, |, ~, parentheses, commas, the quantifiers all/exists and names.

Step 3: Define the grammar by precedence: formula -> disj [-> formula]; disj -> conj {| conj}; conj -> unary {& unary}; unary -> ~unary | quantifier var unary | (formula) | atom; atom -> Predicate(args).

Step 4: Implement one method of the Parser class for every rule, using peek() and eat() to read the tokens, and raise a SyntaxError for invalid input.

Step 5: Convert the AST into a readable formula using the symbols $\forall$, $\exists$, $\neg$, $\wedge$, $\vee$ and $\rightarrow$.

Step 6: Compute the free variables of the formula by ignoring the variables bound by a quantifier.

Step 7: Parse the test formulas, print the AST, the pretty form and the free variables, and stop the program.

CODE:

```python
# Simple First-Order Predicate Calculus (FOPC) parser - recursive descent
import re

TOKEN = re.compile(r"\s*(->|&|\||~|\(|\)|,|(?:all|exists)\b|[A-Za-z_][A-Za-z0-9_]*)")

def tokenize(s):
    toks, pos = [], 0
    while pos < len(s):
        m = TOKEN.match(s, pos)
        if not m: raise SyntaxError(f"Bad character at {pos}: {s[pos]!r}")
        toks.append(m.group(1)); pos = m.end()
    return toks

class Parser:
    def __init__(self, s): self.t, self.i = tokenize(s), 0
    def peek(self): return self.t[self.i] if self.i < len(self.t) else None
    def eat(self, x=None):
        tok = self.peek()
        if tok is None: raise SyntaxError("unexpected end of input")
        if x and tok != x: raise SyntaxError(f"expected {x!r} but found {tok!r}")
        self.i += 1; return tok
    def formula(self):                      # implication (right associative)
        left = self.disj()
        if self.peek() == "->":
            self.eat(); return ("->", left, self.formula())
        return left
    def disj(self):
        left = self.conj()
        while self.peek() == "|":
            self.eat(); left = ("|", left, self.conj())
        return left
    def conj(self):
        left = self.unary()
        while self.peek() == "&":
            self.eat(); left = ("&", left, self.unary())
        return left
    def unary(self):
        tok = self.peek()
        if tok == "~":
            self.eat(); return ("~", self.unary())
        if tok in ("all", "exists"):
            self.eat(); var = self.eat(); return (tok, var, self.unary())
```

```
            if tok == "(":
                self.eat("("); f = self.formula(); self.eat(")"); return f
            return self.atom()
        def atom(self):
            pred = self.eat(); args = []
            if self.peek() == "(":
                self.eat("(")
                args.append(self.eat())
                while self.peek() == ",":
                    self.eat(); args.append(self.eat())
                self.eat(")")
            return ("pred", pred, args)

    def to_str(f):
        k = f[0]
        if k == "pred": return f"{f[1]}({', '.join(f[2])})"
        if k == "~": return f"¬{to_str(f[1])}"
        if k in ("all", "exists"): return f"{'∀' if k=='all' else '∃'}{f[1]}.{to_str(f[2])}"
        sym = {"&": "∧", "|": "∨", "->": "→"}[k]
        return f"({to_str(f[1])} {sym} {to_str(f[2])})"

    def free_vars(f, bound=()):
        k = f[0]
        if k == "pred": return {a for a in f[2] if a[0].islower() and a not in bound}
        if k == "~": return free_vars(f[1], bound)
        if k in ("all", "exists"): return free_vars(f[2], bound + (f[1],))
        return free_vars(f[1], bound) | free_vars(f[2], bound)

    for s in ["all x (Man(x) -> Mortal(x))",
              "exists y (Dog(y) & Loves(John, y))",
              "~Likes(Mary, x) | Happy(Mary)",
              "all x (Student(x) -> exists y (Book(y) & Reads(x, y)))",
              "Man(x) -> ("]:
        print("Input :", s)
        try:
            ast = Parser(s).formula()
            print("AST   :", ast)
            print("Pretty:", to_str(ast))
            print("Free variables:", free_vars(ast) or "none")
        except SyntaxError as e:
            print("Syntax error:", e)
        print()
```

OUTPUT:

```
Input : all x (Man(x) -> Mortal(x))
AST   : ('all', 'x', ('->', ('pred', 'Man', ['x']), ('pred', 'Mortal', ['x'])))
Pretty: ∀x.(Man(x) → Mortal(x))
Free variables: none

Input : exists y (Dog(y) & Loves(John, y))
AST   : ('exists', 'y', ('&', ('pred', 'Dog', ['y']), ('pred', 'Loves', ['John', 'y'])))
Pretty: ∃y.(Dog(y) ∧ Loves(John, y))
Free variables: none

Input : ~Likes(Mary, x) | Happy(Mary)
AST   : ('|', ('~', ('pred', 'Likes', ['Mary', 'x'])), ('pred', 'Happy', ['Mary']))
Pretty: (¬Likes(Mary, x) ∨ Happy(Mary))
Free variables: {'x'}

Input : all x (Student(x) -> exists y (Book(y) & Reads(x, y)))
AST   : ('all', 'x', ('->', ('pred', 'Student', ['x']), ('exists', 'y', ('&', ('pred', 'Book', ['y']), ('pred',
'Reads', ['x', 'y'])))))
Pretty: ∀x.(Student(x) → ∃y.(Book(y) ∧ Reads(x, y)))
Free variables: none

Input : Man(x) -> (
Syntax error: unexpected end of input
```

RESULT:

Four well-formed formulas were parsed into ASTs and printed in logical notation, for example "all x (Man(x) -> Mortal(x))" became ∀x.(Man(x) → Mortal(x)). In "~Likes(Mary, x) | Happy(Mary)" the variable x was correctly reported as free, and the incomplete formula "Man(x) -> (" was rejected with the error "unexpected end of input". Thus the FOPC parser was implemented successfully.

EXPERIMENT NO: 19

WORD SENSE DISAMBIGUATION USING THE LESK ALGORITHM

Date: _____

AIM:

To disambiguate the sense of an ambiguous word in a sentence using the Lesk algorithm, which chooses the WordNet sense whose definition overlaps most with the context.

ALGORITHM:

Step 1: Start the program.

Step 2: Import wordnet from nltk.corpus and define a small set of stop words.

Step 3: For the ambiguous word, collect the context words of the sentence after removing the stop words and the target word.

Step 4: For every WordNet synset of the word, build its signature from the words of its definition and examples.

Step 5: Compute the overlap, i.e. the number of common words, between the signature and the context.

Step 6: Select the sense with the maximum overlap and print its name, overlap score and meaning.

Step 7: Repeat for all test sentences and stop the program.

CODE:

```
# Word Sense Disambiguation using the (simplified) Lesk algorithm
import nltk
from nltk.corpus import wordnet as wn
nltk.download("wordnet", quiet=True)

STOP = {"a","an","the","of","to","in","on","at","and","or","is","was","i","he","she","it","we",
        "for","with","by","that","this","as","from","his","her","my","after","into","up"}

def tokens(text):
    return {w.strip(".,;:()'\"").lower() for w in text.split()} - STOP

def lesk(word, sentence, pos=wn.NOUN):
    context = tokens(sentence) - {word}
    best, best_score = None, -1
    for sense in wn.synsets(word, pos):
        signature = tokens(sense.definition())
        for ex in sense.examples():
            signature |= tokens(ex)
        score = len(signature & context)            # overlap between gloss and context
        if score > best_score:
            best, best_score = sense, score
    return best, best_score

tests = [("bank", "I went to the bank to deposit my money and check the account balance"),
         ("bank", "The fisherman sat on the bank of the river and watched the water"),
         ("mouse","She moved the mouse and clicked the icon on the computer screen"),
         ("mouse","The cat chased the small mouse that was eating grain in the barn"),
         ("plant","The plant employs workers who manufacture cars in the factory")]

for word, sent in tests:
    sense, score = lesk(word, sent)
    print(f"Sentence : {sent}")
    print(f"Word     : {word}")
    print(f"Sense    : {sense.name()}  (overlap = {score})")
    print(f"Meaning  : {sense.definition()}\n")
```

OUTPUT:

```
Sentence : I went to the bank to deposit my money and check the account balance
Word     : bank
Sense    : depository_financial_institution.n.01  (overlap = 2)
Meaning  : a financial institution that accepts deposits and channels the money into lending activities

Sentence : The fisherman sat on the bank of the river and watched the water
Word     : bank
Sense    : bank.n.01  (overlap = 4)
Meaning  : sloping land (especially the slope beside a body of water)

Sentence : She moved the mouse and clicked the icon on the computer screen
Word     : mouse
Sense    : mouse.n.04  (overlap = 2)
Meaning  : a hand-operated electronic device that controls the coordinates of a cursor on your computer screen
as you move it around on a pad; on the bottom of the device is a ball that rolls on the surface of the pad

Sentence : The cat chased the small mouse that was eating grain in the barn
Word     : mouse
Sense    : mouse.n.01  (overlap = 1)
Meaning  : any of numerous small rodents typically resembling diminutive rats having pointed snouts and small
ears on elongated bodies with slender usually hairless tails

Sentence : The plant employs workers who manufacture cars in the factory
Word     : plant
Sense    : plant.n.01  (overlap = 1)
Meaning  : buildings for carrying on industrial labor
```

RESULT:

The Lesk algorithm selected the correct sense in all five test sentences: "bank" as a financial institution in the money sentence and as sloping land in the river sentence, "mouse" as the computer input device (mouse.n.04) and as the rodent (mouse.n.01), and "plant" as a factory building (plant.n.01). Thus word sense disambiguation using Lesk was implemented successfully.

EXPERIMENT NO: 20

INFORMATION RETRIEVAL USING TF-IDF

Date: _____

AIM:

To implement a basic information retrieval system that ranks documents for a user query using TF-IDF (Term Frequency - Inverse Document Frequency) weights and cosine similarity.

ALGORITHM:

Step 1: Start the program.

Step 2: Store a collection of five documents in a list.

Step 3: Create a TfidfVectorizer (with English stop words removed) and call fit_transform() to obtain the TF-IDF matrix of the documents.

Step 4: Print the size of the vocabulary and the three highest weighted terms of every document.

Step 5: For a query, convert it into a TF-IDF vector with transform() and compute its cosine similarity with all the documents.

Step 6: Sort the documents in decreasing order of similarity and print the top-ranked documents with their scores.

Step 7: Stop the program.

CODE:

```
# Basic Information Retrieval system using TF-IDF and cosine similarity
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = [
    "Machine learning is a method of data analysis that automates model building.",
    "Natural language processing enables computers to understand human language.",
    "Deep learning uses neural networks with many layers to learn from data.",
    "The cricket match was postponed due to heavy rain in Chennai.",
    "Information retrieval ranks documents by their relevance to a user query.",
]

vectorizer = TfidfVectorizer(stop_words="english")
tfidf = vectorizer.fit_transform(documents)
terms = vectorizer.get_feature_names_out()
print("Vocabulary size:", len(terms), "| TF-IDF matrix shape:", tfidf.shape)

# highest weighted terms of every document
for i, row in enumerate(tfidf.toarray()):
    top = row.argsort()[::-1][:3]
    print(f"Doc {i+1} top terms:", [(terms[j], round(float(row[j]), 3)) for j in top])

def search(query, k=3):
    q = vectorizer.transform([query])
    scores = cosine_similarity(q, tfidf).ravel()
    ranking = scores.argsort()[::-1][:k]
    print(f"\nQuery: '{query}'")
    for rank, i in enumerate(ranking, 1):
        print(f"  Rank {rank}: Doc {i+1}  score = {scores[i]:.4f}  | {documents[i]}")

search("learning from data with neural networks")
search("ranking documents for a user query")
search("understanding human language by computers")
```

OUTPUT:

```
Vocabulary size: 34 | TF-IDF matrix shape: (5, 34)
Doc 1 top terms: [('machine', 0.37), ('method', 0.37), ('building', 0.37)]
Doc 2 top terms: [('language', 0.632), ('understand', 0.316), ('computers', 0.316)]
Doc 3 top terms: [('uses', 0.37), ('networks', 0.37), ('deep', 0.37)]
Doc 4 top terms: [('rain', 0.408), ('postponed', 0.408), ('cricket', 0.408)]
Doc 5 top terms: [('user', 0.378), ('retrieval', 0.378), ('documents', 0.378)]

Query: 'learning from data with neural networks'
  Rank 1: Doc 3  score = 0.6725  | Deep learning uses neural networks with many layers to learn from data.
  Rank 2: Doc 1  score = 0.2651  | Machine learning is a method of data analysis that automates model building.
  Rank 3: Doc 5  score = 0.0000  | Information retrieval ranks documents by their relevance to a user query.

Query: 'ranking documents for a user query'
  Rank 1: Doc 5  score = 0.6547  | Information retrieval ranks documents by their relevance to a user query.
  Rank 2: Doc 4  score = 0.0000  | The cricket match was postponed due to heavy rain in Chennai.
  Rank 3: Doc 3  score = 0.0000  | Deep learning uses neural networks with many layers to learn from data.

Query: 'understanding human language by computers'
  Rank 1: Doc 2  score = 0.7303  | Natural language processing enables computers to understand human language.
  Rank 2: Doc 5  score = 0.0000  | Information retrieval ranks documents by their relevance to a user query.
  Rank 3: Doc 4  score = 0.0000  | The cricket match was postponed due to heavy rain in Chennai.
```

RESULT:

The vectorizer produced a 5 x 34 TF-IDF matrix. For the three queries, the most relevant documents were retrieved at rank 1 with scores 0.6725 (Doc 3, neural networks), 0.6547 (Doc 5, information retrieval) and 0.7303 (Doc 2, natural language processing), while unrelated documents received a score of 0. Thus a basic TF-IDF information retrieval system was implemented successfully.

EXPERIMENT NO: 21

SYNTAX-DRIVEN SEMANTIC ANALYSIS

Date: _____

AIM:

To perform syntax-driven semantic analysis by extracting the noun phrases of a sentence, finding their meanings in WordNet and building a simple predicate-argument representation from the dependency parse.

ALGORITHM:

Step 1: Start the program.

Step 2: Import spacy, nltk and wordnet; load the model en_core_web_sm and download the wordnet data.

Step 3: Parse the text with spaCy and split it into sentences.

Step 4: Extract the noun phrases of every sentence with sent.noun_chunks and print the head word and its grammatical role (dependency label).

Step 5: Look up the head lemma in WordNet and print the definition of its first noun synset as the meaning of the phrase.

Step 6: For every verb, collect its subject (agent), object (theme) and prepositional object (location/goal) from the dependency tree.

Step 7: Print the semantic form verb(agent, theme, location) and stop the program.

CODE:

```
# Syntax-driven semantic analysis: noun phrases, their meanings and predicate-argument structure
import spacy, nltk
from nltk.corpus import wordnet as wn
nltk.download("wordnet", quiet=True)
nlp = spacy.load("en_core_web_sm")

text = "The quick brown fox jumps over the lazy dog. A young scientist built a powerful robot."
doc = nlp(text)

def meaning(chunk):
    syn = wn.synsets(chunk.root.lemma_, pos=wn.NOUN)
    return syn[0].definition() if syn else "no WordNet entry"

for sent in doc.sents:
    print("Sentence:", sent.text)
    for nc in sent.noun_chunks:
        print(f"  NP   : {nc.text:<24} head={nc.root.text:<10} role={nc.root.dep_}")
        print(f"         meaning -> {meaning(nc)}")
    for tok in sent:
        if tok.pos_ == "VERB":
            agent = [c.text for c in tok.children if c.dep_ == "nsubj"]
            theme = [c.text for c in tok.children if c.dep_ in ("dobj", "obj")]
            place = [g.text for c in tok.children if c.dep_ == "prep" for g in c.children if g.dep_ == "pobj"]
            print(f"  Semantics: {tok.lemma_}(agent={agent}, theme={theme}, location/goal={place})")
    print()
```

OUTPUT:

```
Sentence: The quick brown fox jumps over the lazy dog.
  NP   : The quick brown fox      head=fox        role=nsubj
         meaning -> alert carnivorous mammal with pointed muzzle and ears and a bushy tail; most are predators
that do not hunt in packs
  NP   : the lazy dog             head=dog        role=pobj
         meaning -> a member of the genus Canis (probably descended from the common wolf) that has been
domesticated by man since prehistoric times; occurs in many breeds
  Semantics: jump(agent=['fox'], theme=[], location/goal=['dog'])

Sentence: A young scientist built a powerful robot.
  NP   : A young scientist        head=scientist  role=nsubj
         meaning -> a person with advanced knowledge of one or more sciences
  NP   : a powerful robot         head=robot      role=dobj
         meaning -> a mechanism that can move automatically
  Semantics: build(agent=['scientist'], theme=['robot'], location/goal=[])
```

RESULT:

The noun phrases "The quick brown fox", "the lazy dog", "A young scientist" and "a powerful robot" were extracted along with their WordNet meanings. The semantic forms jump(agent=fox, location/goal=dog) and build(agent=scientist, theme=robot) were derived from the syntax. Thus syntax-driven semantic analysis was implemented successfully.

EXPERIMENT NO: 22

REFERENCE RESOLUTION

Date: _____

AIM:

To perform reference (pronoun) resolution in a text by linking each pronoun to its antecedent using gender, number, animacy and recency/salience constraints.

ALGORITHM:

Step 1: Start the program.

Step 2: Load the spaCy model and define small gender lists for names and a table that gives the gender/number of the pronouns he, she, it, they, etc.

Step 3: Extract all the non-pronoun noun phrases of the text as candidate antecedents.

Step 4: For every pronoun, keep only the candidates that occur before it and agree with it in gender and number.

Step 5: Score each remaining candidate: +2 if it is a subject, minus the number of sentences between the candidate and the pronoun, minus a small word-distance penalty.

Step 6: Choose the candidate with the highest score as the antecedent.

Step 7: Print a table of pronouns with their antecedents and the text with the resolved references; stop the program.

CODE:

```
# Reference (pronoun) resolution using gender, number, animacy and recency/salience rules
import spacy
nlp = spacy.load("en_core_web_sm")

MALE, FEMALE = {"john", "david", "raj", "tom"}, {"mary", "anita", "lisa", "priya"}
PRONOUNS = {"he": "m", "him": "m", "his": "m", "she": "f", "her": "f", "hers": "f",
            "it": "n", "its": "n", "they": "pl", "them": "pl", "their": "pl"}

def gender(span):
    w = span.root.text.lower()
    if w in MALE: return "m"
    if w in FEMALE: return "f"
    return "n"                      # things / non-person nouns

def plural(span): return span.root.tag_ in ("NNS", "NNPS")

def resolve(text):
    doc = nlp(text)
    mentions = [c for c in doc.noun_chunks if c.root.pos_ != "PRON"]
    sent_no = {s.start: n for n, s in enumerate(doc.sents)}
    result = {}
    for tok in doc:
        key = tok.text.lower()
        if tok.pos_ != "PRON" or key not in PRONOUNS: continue
        need, best, best_score = PRONOUNS[key], None, -99
        for m in mentions:
            if m.end > tok.i: continue                      # antecedent must come first
            ok = plural(m) if need == "pl" else (not plural(m) and gender(m) == need)
            if not ok: continue
            dist = sent_no[tok.sent.start] - sent_no[m.sent.start]       # sentences apart
            score = (2 if m.root.dep_ == "nsubj" else 0) - dist - (tok.i - m.end) / 100
            if score > best_score: best, best_score = m, score
        result[tok.i] = best
    return doc, result

text = ("John met Mary at the station. He gave her a book. The book was old, but it was interesting. "
        "Mary thanked him. The children opened the box and they laughed.")
doc, res = resolve(text)
print(text, "\n")
print(f"{'Pronoun':<10}{'Position':<10}Antecedent")
print("-" * 40)
for i, ant in res.items():
    print(f"{doc[i].text:<10}{i:<10}{ant.text if ant else '(not found)'}")

out = [f"{t.text}[{res[t.i].text}]" if t.i in res and res[t.i] else t.text for t in doc]
print("\nResolved text:\n", " ".join(out))
```

OUTPUT:

```
John met Mary at the station. He gave her a book. The book was old, but it was interesting. Mary thanked him.
The children opened the box and they laughed.

Pronoun   Position  Antecedent
----------------------------------------
He        7         John
her       9         Mary
it        19        The book
him       25        John
they      33        The children

Resolved text:
 John met Mary at the station . He[John] gave her[Mary] a book . The book was old , but it[The book] was
interesting . Mary thanked him[John] . The children opened the box and they[The children] laughed .
```

RESULT:

All five pronouns of the text were resolved correctly: He -> John, her -> Mary, it -> The book, him -> John and they -> The children. The resolved text was printed with the antecedent in brackets after every pronoun. Thus reference resolution was implemented successfully.

EXPERIMENT NO: 23

COHERENCE EVALUATION OF A TEXT

Date: _____

AIM:

To evaluate the coherence of a text by measuring the lexical cohesion between adjacent sentences together with the use of discourse connectives and pronouns.

ALGORITHM:

Step 1: Start the program.

Step 2: Load the spaCy model and define the lists of discourse connectives and pronouns.

Step 3: Split the text into sentences and represent each sentence by the lemmas of its nouns, verbs and adjectives.

Step 4: Compute the TF-IDF vectors of the sentences and the cosine similarity between every pair of adjacent sentences (lexical cohesion).

Step 5: Check whether the following sentence begins a link with a connective or a pronoun (1 if present, otherwise 0).

Step 6: Compute coherence score = 0.7 x (mean lexical similarity) + 0.3 x (mean link value).

Step 7: Classify the text as COHERENT if the score is at least 0.3, otherwise INCOHERENT; print the result for both sample texts and stop.

CODE:

```
# Evaluate the coherence of a text (lexical cohesion + connectives + pronoun continuity)
import spacy
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
nlp = spacy.load("en_core_web_sm")

CONNECTIVES = {"because", "therefore", "however", "moreover", "then", "so", "but", "thus",
               "since", "also", "finally", "hence", "although"}
PRONOUNS = {"he", "she", "it", "they", "this", "these", "those", "him", "her", "them"}

def coherence(text, verbose=True):
    sents = list(nlp(text).sents)
    lem = [" ".join(t.lemma_.lower() for t in s if t.pos_ in ("NOUN", "PROPN", "VERB", "ADJ")) for s in sents]
    X = TfidfVectorizer().fit_transform(lem)
    sims, links = [], []
    for i in range(len(sents) - 1):
        sim = cosine_similarity(X[i], X[i + 1])[0, 0]
        nxt = {t.text.lower() for t in sents[i + 1]}
        link = int(bool(nxt & CONNECTIVES) or bool(nxt & PRONOUNS))
        sims.append(sim); links.append(link)
        if verbose:
            print(f"  S{i+1}->S{i+2}: lexical overlap = {sim:.3f}, connective/pronoun link = {link}")
    score = 0.7 * sum(sims) / len(sims) + 0.3 * sum(links) / len(links)
    return score

coherent = ("Rahul bought a new laptop yesterday. The laptop has a fast processor, so it runs programs quickly.
"
            "However, the laptop battery drains fast. Rahul therefore carries a charger everywhere.")
incoherent = ("Rahul bought a new laptop yesterday. The monsoon arrives in June over Kerala. "
              "Penguins live near the polar regions. Quantum computers need extreme cooling.")

for name, txt in [("Coherent text", coherent), ("Incoherent text", incoherent)]:
    print(f"{name}:\n  {txt}")
    s = coherence(txt)
    label = "COHERENT" if s >= 0.3 else "INCOHERENT"
    print(f"  Coherence score = {s:.3f}  ->  {label}\n")
```

OUTPUT:

```
Coherent text:
  Rahul bought a new laptop yesterday. The laptop has a fast processor, so it runs programs quickly. However,
the laptop battery drains fast. Rahul therefore carries a charger everywhere.
  S1->S2: lexical overlap = 0.087, connective/pronoun link = 1
  S2->S3: lexical overlap = 0.113, connective/pronoun link = 1
  S3->S4: lexical overlap = 0.000, connective/pronoun link = 1
  Coherence score = 0.347  ->  COHERENT

Incoherent text:
  Rahul bought a new laptop yesterday. The monsoon arrives in June over Kerala. Penguins live near the polar
regions. Quantum computers need extreme cooling.
  S1->S2: lexical overlap = 0.000, connective/pronoun link = 0
  S2->S3: lexical overlap = 0.000, connective/pronoun link = 0
  S3->S4: lexical overlap = 0.000, connective/pronoun link = 0
  Coherence score = 0.000  ->  INCOHERENT
```

RESULT:

The coherent paragraph about a laptop obtained a coherence score of 0.347 and was classified as COHERENT, because its sentences share words and are joined by connectives such as "so", "however" and "therefore". The paragraph of unrelated sentences obtained a score of 0.000 and was classified as INCOHERENT. Thus the coherence of text was evaluated successfully.

EXPERIMENT NO: 24

DIALOG ACT RECOGNITION

Date: _____

AIM:

To recognise the dialog act (greeting, question, request, thanks, apology, etc.) of every utterance in a conversation using rule-based pattern matching.

ALGORITHM:

Step 1: Start the program.

Step 2: Import the re module.

Step 3: Define an ordered list of (dialog act, regular expression) rules for greeting, goodbye, thanks, apology, request, yes/no question, wh-question, agreement, rejection and command.

Step 4: Define dialog_act(utterance) that converts the utterance to lower case and returns the act of the first rule that matches, or STATEMENT if no rule matches.

Step 5: Store a sample dialog with the speaker, the utterance and the expected (gold) act.

Step 6: Predict the act of each utterance, print it next to the gold act and count the correct predictions.

Step 7: Print the accuracy and stop the program.

CODE:

```python
# Dialog act recognition using rule-based (regular-expression) classification
import re

RULES = [
    ("GREETING",    r"^(hi|hello|hey|good (morning|afternoon|evening))\b"),
    ("GOODBYE",     r"\b(bye|goodbye|see you|take care)\b"),
    ("THANKS",      r"\b(thanks|thank you|thx)\b"),
    ("APOLOGY",     r"\b(sorry|apologi[sz]e|my bad)\b"),
    ("REQUEST",     r"^(please|could you|can you|would you|kindly)\b|\bplease\b"),
    ("YES-NO-QUESTION", r"^(is|are|do|does|did|can|will|would|have|has|should)\b.*\?$"),
    ("WH-QUESTION", r"^(what|where|when|why|who|whom|which|how)\b"),
    ("AGREEMENT",   r"^(yes|yeah|sure|ok|okay|right|exactly|of course)\b"),
    ("REJECTION",   r"^(no|nope|never|not really)\b"),
    ("COMMAND",     r"^(open|close|book|show|tell|send|call|find|play)\b"),
]

def dialog_act(utt):
    u = utt.lower().strip()
    for act, pattern in RULES:
        if re.search(pattern, u):
            return act
    return "STATEMENT"

dialog = [
    ("A", "Hello, good morning!",                       "GREETING"),
    ("B", "Good morning. How can I help you?",          "GREETING"),
    ("A", "Can you book a table for two tonight?",      "REQUEST"),
    ("B", "Sure, what time would you prefer?",          "AGREEMENT"),
    ("A", "Around 8 pm please.",                        "REQUEST"),
    ("B", "Do you want a vegetarian menu?",             "YES-NO-QUESTION"),
    ("A", "No, any menu is fine.",                      "REJECTION"),
    ("B", "Sorry, 8 pm is fully booked.",               "APOLOGY"),
    ("A", "I will come at 9 pm then.",                  "STATEMENT"),
    ("B", "Your table is confirmed.",                   "STATEMENT"),
    ("A", "Thank you so much!",                         "THANKS"),
    ("B", "Goodbye, see you tonight.",                  "GOODBYE"),
]

print(f"{'Spk':<4}{'Utterance':<42}{'Predicted act':<18}{'Gold'}")
print("-" * 80)
correct = 0
for spk, utt, gold in dialog:
    pred = dialog_act(utt); correct += pred == gold
    print(f"{spk:<4}{utt:<42}{pred:<18}{gold} {'' if pred == gold else '  <-- mismatch'}")
print(f"\nAccuracy: {correct}/{len(dialog)} = {correct/len(dialog):.0%}")
```

OUTPUT:

```
Spk Utterance                                 Predicted act     Gold
--------------------------------------------------------------------------------
A   Hello, good morning!                      GREETING          GREETING
B   Good morning. How can I help you?         GREETING          GREETING
A   Can you book a table for two tonight?     REQUEST           REQUEST
B   Sure, what time would you prefer?         AGREEMENT         AGREEMENT
A   Around 8 pm please.                       REQUEST           REQUEST
B   Do you want a vegetarian menu?            YES-NO-QUESTION   YES-NO-QUESTION
A   No, any menu is fine.                     REJECTION         REJECTION
B   Sorry, 8 pm is fully booked.              APOLOGY           APOLOGY
A   I will come at 9 pm then.                 STATEMENT         STATEMENT
B   Your table is confirmed.                  STATEMENT         STATEMENT
A   Thank you so much!                        THANKS            THANKS
B   Goodbye, see you tonight.                 GOODBYE           GOODBYE

Accuracy: 12/12 = 100%
```

RESULT:

The dialog acts of all 12 utterances of the restaurant-booking conversation were identified correctly (accuracy 12/12 = 100%). Thus dialog act recognition was implemented successfully using rule-based classification.

EXPERIMENT NO: 25

TEXT GENERATION USING GPT-3

Date: _____

AIM:

To generate text for a given prompt using a GPT-3 family language model through the OpenAI Python library.

ALGORITHM:

Step 1: Start the program.

Step 2: Install the OpenAI library using !pip install openai and import OpenAI.

Step 3: Create an API key at the OpenAI platform, store it in Colab Secrets as OPENAI_API_KEY and read it with userdata.get().

Step 4: Create a client object with OpenAI(api_key=...).

Step 5: Define the prompt and call client.completions.create() with the model name, prompt, max_tokens and temperature.

Step 6: Read the generated text from response.choices[0].text and print it with the number of tokens used.

Step 7: Stop the program.

CODE:

```
# Text generation with GPT-3 using the OpenAI Python library
# In Colab first run:  !pip install openai
import os
from openai import OpenAI
from google.colab import userdata            # store your key in Colab "Secrets" as OPENAI_API_KEY

client = OpenAI(api_key=userdata.get("OPENAI_API_KEY"))

prompt = "Write a short paragraph about the importance of Natural Language Processing."

# GPT-3 family completion model (text-davinci-003 is retired; gpt-3.5-turbo-instruct is its successor)
response = client.completions.create(
    model="gpt-3.5-turbo-instruct",
    prompt=prompt,
    max_tokens=120,
    temperature=0.7,
)

print("Prompt :", prompt)
print("\nGenerated text:\n", response.choices[0].text.strip())
print("\nTokens used:", response.usage.total_tokens)
```

OUTPUT:

Output screenshot to be added: this program calls the paid OpenAI API and needs your own API key, so run the cell in Google Colab (after adding OPENAI_API_KEY in Colab Secrets) and paste the screenshot here.

RESULT:

The program sends a prompt to the GPT-3 family completion model (gpt-3.5-turbo-instruct, the successor of the retired text-davinci-003) using the OpenAI library, and the text generated by the model for the prompt is printed. Thus text generation using GPT-3 was implemented.

EXPERIMENT NO: 26

MACHINE TRANSLATION USING HUGGING FACE TRANSFORMERS

Date: _____

AIM:

To translate English text into French using a pre-trained neural machine translation model from the Hugging Face Transformers library.

ALGORITHM:

Step 1: Start the program.

Step 2: Install the libraries using !pip install transformers sentencepiece and import pipeline from transformers.

Step 3: Create a translation pipeline with pipeline("translation_en_to_fr", model="Helsinki-NLP/opus-mt-en-fr"), which downloads the pre-trained model.

Step 4: Store the English sentences to be translated in a list.

Step 5: Pass each sentence to the translator and read the translation_text field of the result.

Step 6: Print every English sentence with its French translation.

Step 7: Stop the program.

CODE:

```
# English -> French machine translation with Hugging Face Transformers
# In Colab first run:  !pip install transformers sentencepiece
from transformers import pipeline

translator = pipeline("translation_en_to_fr", model="Helsinki-NLP/opus-mt-en-fr")

sentences = [
    "Natural language processing helps computers understand human language.",
    "I am learning machine translation in my laboratory class.",
    "Where is the nearest railway station?",
]

for s in sentences:
    result = translator(s, max_length=100)
    print("English :", s)
    print("French  :", result[0]["translation_text"], "\n")
```

OUTPUT:

> *Output screenshot to be added: the pre-trained model is downloaded from the Hugging Face Hub when the cell runs, so run the cell in Google Colab and paste the screenshot here.*

RESULT:

The pre-trained MarianMT model (Helsinki-NLP/opus-mt-en-fr) was used through the Transformers pipeline, and every English sentence was translated into French and printed. Thus machine translation from English to French was implemented.